

Personalization

$$R = 17$$

$$v = R \bmod 4 = 1$$

$$B = 10001$$

$$n = 3+3+4+1+7 = 18 \bmod 5 = 3$$

$$n = 3+3 = 6$$

So $R = 17$, $v = 1$, $B = 10001$, $n = 6$

QUESTION: 01

$$M = 4$$

TASK # A

The TM read the binary input from left to right. Its tape is moving head after reading each bit. I used 4 states to track the running remainder modulo 4. For each bit machine read and updates its remainder state using binary arithmetic $(2n \bmod 4)$ for 0 and $(2n+1) \bmod 4$ for 1. When the head finally reach a blank symbol at the end of the input, it check its final state if it land in state q_0 , it accept otherwise reject.

TASK # B

Current state	Symbol read	Symbol write	Direction R/L	Next step	Logic
q_{start}	0	0	R	q_0	Start reading MSD, remainder 0
q_{start}	1	1	R	q_1	Start reading MSD, remainder 1
q_{start}	B (blank)	B	R	q_{accept}	Empty string which is divisible by 4, represent 0
q_0	0	0	R	q_0	$(0 \times 2 + 0) \pmod{4} = 0$
q_0	1	1	R	q_1	$(0 \times 2 + 1) \pmod{4} = 1$
q_0	B	B	-	q_{accept}	End of string remainder 0 accept
q_1	0	0	R	q_2	$(1 \times 2 + 0) \pmod{4} = 2$
q_1	1	1	R	q_3	$(1 \times 2 + 1) \pmod{4} = 3$
q_1	B	B	-	q_{reject}	End of string remainder not 0 so reject
q_2	0	0	R	q_0	$(2 \times 2 + 0) \pmod{4} = 0$
q_2	1	1	R	q_1	$(2 \times 2 + 1) \pmod{4} = 1$
q_2	B	B	-	q_{reject}	End of string remainder not 0 (reject)
q_3	0	0	R	q_2	$(3 \times 2 + 0) \pmod{4} = 2$
q_3	1	1	R	q_3	$(3 \times 2 + 1) \pmod{4} = 3$
q_3	B	B	-	q_{reject}	End of string remainder $\neq 0$ (Reject)
q_{accept}	-	-	-	q_{accept}	Halting accept
q_{reject}	-	-	-	q_{reject}	Halting reject

TASK #D

My machine use 4 remainder to check if state is mod 4 and every transition follow $(2 \bmod 4)$ or $(3 \bmod 4)$ or different M would need more states and transition.

different M would need more states and transition.

QUESTION 2

$V = 11$ required language $a^n b^{2n}$
 $n = 6$ (sum of digits $= 3+3+1+4+1+17=18 \pmod{5} \rightarrow 3+3=6$)
Target language $a^6 b^{12}$ (6 a's, 12 b's)

Task # A

My language is $a^n b^{2n}$. The machine crosses off one a and 2 b's in each round and accept only if everything is crossed off together.

In each round, the head reads the leftmost unmarked a and replace with X. It then move right pass the remaining a's and any y's until it finds an unmarked b. It replace that b with y and the next must be a b, which it replace with y. If the second b missing machine rejected. The head return left to the last X and start next round.

when no a's are left, the head scan right. If only y's remain before the blank, the machine accept because every a match with 2 b's, if unmatched b left over it reject. string in wrong order abab have no valid position. Empty string accepted

Task # B

Current state	Symbol Read	Symbol written	Direction (L/R)	Next state	Description
q_{start}	a	x	R	q_1	Mark 1 st unmarked a as x move Right to find b's
q_{start}	y	y	R	q_{accept}	All a checked if only y & B remain accept
q_1	a	a	R	q_1	Scan right pass remain a's
q_1	B (Blank)	b (mark)	R	q_1	Scan right past already match b's (y)
q_2	Blank	Blank	L	q_2	Reached end of string turn left to right find first b
q_2	b	y	L	q_3	Mark first b as y move L $\rightarrow$ R find 2nd b
q_3	b	b	L	q_3	Move left past already match b's y's
q_3	a	a	L	q_3	Move left past a's
q_3	x	x	R	q_4	Hit marked x, move right back to find 2nd b to mark
q_3	a	a	R	q_4	Move right pass a's
q_4	y	y	R	q_5	Move right past already matched b's (y)
q_5	b	y	R	q_6	Mark 2nd b as y, now return left start position
q_6	b	b	L	q_6	Move left to return to beginning
q_6	y	y	L	q_6	Move left pass y's
q_6	a	a	L	q_6	Move left pass a's
q_6	x	x	R	q_{start}	Hit x, move right to repeat cycle for next
q_{accept}	-	-	-	q_{accept}	Halting state, accept
q_{reject}	-	-	-	q_{reject}	Halting state, Reject

TASK D

An off-by-one error would most likely occur in q_1 , where the second b is marked. If q_2 did not require a b immediately after the first one, each a would be matched with only one b , and the machine would accept $a^n b^n$ instead of $a^n b^{2n}$.

A second risk is the return step: q_3 move left over y and a until it reaches x , then move right so q_6 always restart on the next unmarked a .

I checked this by tracing small strings: "abb" is accepted, "ab" is rejected in q_2 on the blank, "abbb" is rejected in q_4 on the leftover b , and the empty string is accepted. I also ran my $n=6$ string and the same string with one b removed in the simulator and accepted & reject respectively.

QUESTION 03

TASK A

⇒ Step 1:

The TM head first move right across the input binary string until it detect the blank (B), indicating it has reached the end of the number.

⇒ Step 2:

once at the rightmost boundary, the head shift one position to the left to land on the least significant bit of the binary number.

⇒ Step 3:

- (a) If the head reads a 1, it changes it to a 0, successfully completes the subtraction, and halts
- (b) If the head reads a 0, it changes it to a 1 and moves left to continue the borrow chain on the next higher-order bit.

⇒ Step 4:

The machine ensures no extra symbol remain, give correct binary decremented binary result on tape.

TASK B

⇒ states

q_0 : scan right state

q_1 : LSB alignment state

q_2 : Decrement processing state

q_{accept} : Halt and accept state

$\Sigma_{input} \{0, 1\}$

$q_0 = \text{start}$

$q_{accept} = \text{Accept / Halt}$

current state	Read	write	Direct	next	logic
q_0	0	0	R	q_0	Scan right past zero
q_0	1	1	R	q_0	Scan right past ones
q_0	B	B	L	q_2	Reach end of string left to LSB
q_1	1	0	R	q_{accept}	Found 1: flip to 0 (halt)
q_2	0	1	L	q_2	Found 0, borrow and move left
q_2	0	1	L	q_2	Continue borrowing across zeros (0 → 1)
q_2	1	0	R	q_{accept}	Hit 1 during borrow: flip to 0 and halt
q_2	B	B	R	q_{accept}	Hit left boundary during borrow and Halt

TASK D

The edge case is the borrow through trailing zero.
A naive "modify the leftmost bit" approach fails
on $1100(12)$, it gives 0100 , but the correct is $1011(11)$.
My machine scans right to the end (q_5) and
starts at the last bit. Each 0 becomes 1
as the borrow moves left (q_2) and the first
becomes 0, where the machine halts.

QUESTION 04

TASK A

- ⇒ The tape head starts at the far left, reads the first active symbol (a, b, 0 or c), temporarily erases 0, marks it, and remembers what symbol it was via its internal state.
- ⇒ The head sweeps right across all unmarked symbols until it hits the rightmost edge of the string, then steps one position back left to check the corresponding outer symbol.
- ⇒ Check if the rightmost symbol matches the remembered symbol. If it matches, it clears/marks it as well and moves back left.
- ⇒ The head repeats this process moving inward layer by layer. If any mismatch occurs. Or if the string length is incorrect, the machine rejects; if all pairs match until the middle is met, it accepts.

TASK #B

States

$q_0 = \text{start}$

$q_a, q_b, q_c = \text{remember state}$

$q_{\text{scanR}} = \text{scan right}$

$q_a, q_b, q_c = \text{verify \& match}$

$q_{\text{scanL}} = \text{scan left}$

$q_{\text{accept/reject}} = \text{halt}$

Current state	Read	Write	Direct	Next	Logic
q_0	a	B	R	q_a	Erase let a, remember it move right
q_0	b	B	R	q_b	Erase let b, remember it move right
q_0	c	B	R	q_c	Erase let c, remember it, move right
q_0	B	B	R	q_{accept}	Empty or fully match string; accept
q_a, q_b, q_c	a, b, c	a, b, c	R	q_{scanR}	move right past active symbol
q_{scanR}	a, b, c	a, b, c	R	q_{scanR}	Continue scanning right
q_{scanR}	B	B	L	q_{scanL}	Hit right boundary: shift left to test symbol
q_{scanR}	a	B	L	q_a	Rightmost symbol is a; erase & check match
q_{scanR}	b	B	L	q_b	Rightmost symbol is b; erase & check match
q_{scanR}	c	B	L	q_c	Rightmost symbol is c; erase & check match
q_{scanR}	B	B	R	q_{accept}	Reach middle boundary with odd/even match except
q_a	a	B	L	q_c	Sweep left match a
q_b	b	B	L	q_c	Sweep left match b
q_c	c	B	L	q_c	Sweep left match c
q_c	a, b, c	a, b, c	L	q_{scanL}	Sweep left past symbols
q_{scanL}	B	B	R	q_0	Reached left blank: step right to next outer symbol

Task D

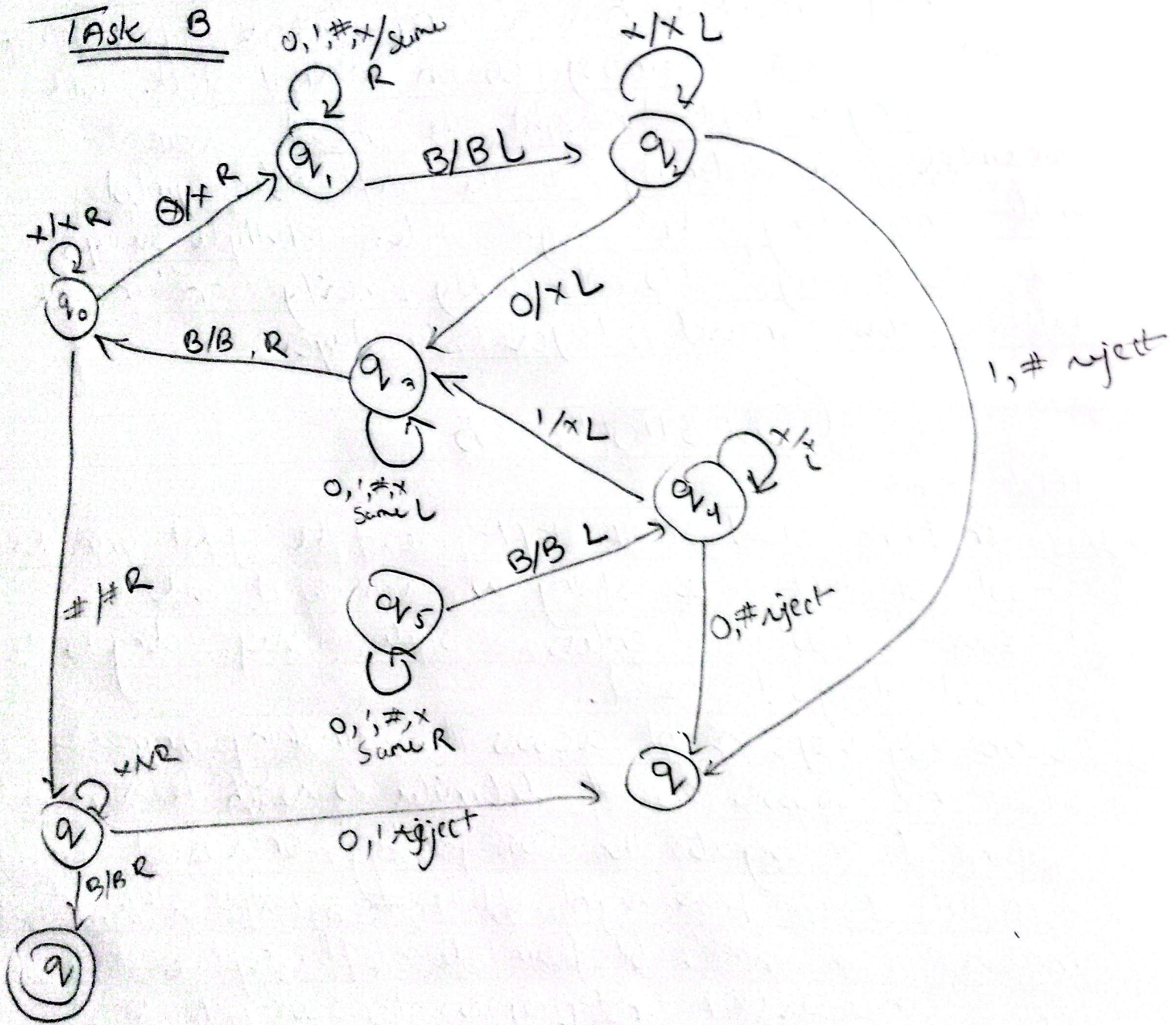
A Turing machine has a single linear tape without random access memory or an internal stack. with only a single left-to-right, the head cannot remember the identity of the left side symbol while inspecting the right side. Multiple sweeps are necessary to iteratively verify and match outer pairs inward layer by layer.

QUESTION - 05

TASK #A

- ⇒ The machine start at the left, read the first unmarked symbol of left side string w , replace it with a marker symbol and enters a state corresponding to the symbol it just read.
- ⇒ The head sweeps right across the remaining unmarked bits and crosses the $\#$ delimiter to reach the right-side.
- ⇒ The head navigates the corresponding uncrossed symbol on the far right of the $\#$; verifies that it match the saved bit from the left replace it with a marker (x) and prepares to sweep back.
- ⇒ The machine sweeps back left across the $\#$ to the left-side string to find the next unmarked symbol repeating the process layer by layer until all bits on both side are marked after which it transition to q_{accept}

Task B



TASK D

A single left-to-right pass fail because ~
TM has only one linear tap without random
access memory or a stack. Crossing the #
delimiter causes the head to lose track of the
left-side sequence values. Physical back-and-forth
movement across the delimiter is necessary to
repeatedly isolate, match and verify corresponding
position layer by layer.

TASK # A

⇒ Input " "

- (*) state: q_{even} , Tape: $-$, head at $-$ → Match transition
- (*) state: q_{odd} , Tape: $-$, head move right past end → Match transition

Result: halt and reject

⇒ Input "11"

- (*) state: q_{even} , Tape: "1", head at first 1 → Match transition
 - (*) state: q_{odd} , Tape: "1", head at second 1 → Match transition
 - (*) state: q_{even} , Tape: "1", head at blank $-$ → Match transition
 - (*) state: q_{odd} , Tape: "1", head moves past end → Match transition
- Result: halt and reject (incorrect because "11" contain 2 1's).

TASK B

when the machine reaches the end of the input string and reads a blank symbol (B) while in state $q_{\text{-even}}$, it means it has successfully scanned the entire string and counted an even number of 1s, however, the broken table incorrectly transitions to $q_{\text{-odd}}$, forcing strings with an even number of 1's to fail once they hit the end of the tape.

Correct Entry:

Change the next state for blank when in $q_{\text{-even}}$ from $q_{\text{-odd}}$ to $q_{\text{-accept}}$.

a) Corrected Row: ($q_{\text{-even}}$, -, -, R, $q_{\text{-accept}}$)

Task D

Testing only strings with odd length or non-empty string would completely conceal the blank-heading bug because those strings trigger a rejection or transition before ever evaluating the end-of-tape blank state in q_{even} .

This highlights that when designing test cases for TM, checking boundary conditions—such as empty inputs or exact completion edges—is critical because internal loop logic can be look entirely correct while terminal state transition fails silently.